



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Certified Kubernetes Application Developer (CKAD)

TGS Ref No: TGS-2025053212

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	30 June 2026	First version — step-by-step guide to all 30 CKAD labs across five domains (Application Design & Build, Application Deployment, Observability & Maintenance, Configuration & Security, Services & Networking); MD and DOCX generated from one source	Tertiary Infotech Academy Pte Ltd

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

Welcome! This guide walks you command-by-command through every hands-on lab in the WSQ course **Certified Kubernetes Application Developer (CKAD)** (Course Code: TGS-2025053212). Over 3½ days you work through all five CKAD exam domains — from building container images, through deploying and updating applications with Helm and Kustomize, observing and securing workloads, to networking Pods with Services and NetworkPolicy.

Work through the labs in order: each builds on the last. Whenever you see a **Test it** box, stop and confirm the result before moving on. All labs run in the browser on Killercodea — the link is at the top of each lab.

Note: Course flow at a glance — **Day 1 (Domain 1):** Container images, Pods, Jobs, CronJobs, multi-container Pods, volumes (Labs 1–8). **Day 2 (Domain 2):** Deployments, release strategies, Helm, Kustomize, DaemonSet & StatefulSet (Labs 9–15). **Day 3 (Domains 3 & 4):** Probes, logging, metrics, debugging, API versions, ConfigMaps, Secrets, SecurityContext, ServiceAccounts, RBAC, Quotas (Labs 16–26). **Day 4 — half day (Domain 5):** Services & DNS, Ingress & TLS, NetworkPolicy (Labs 27–30) then mock-exam practice and final assessment from 2:00 PM.

0. Before You Start — Setup & Prerequisites

0.1 What you need

Tool	Used for	Where to get it
kubectl	All labs — the Kubernetes CLI	kubernetes.io/docs/tasks/tools/ (or pre-installed on Killercodea)
A Kubernetes cluster	Running all 30 labs	Killercodea playground (browser) · minikube · kind · Docker Desktop K8s
Docker CLI / Engine	Labs 1–2 (container image builds)	docker.com (or use Killercodea's pre-installed Docker)
Helm 3	Lab 13 (Helm chart deployment)	helm.sh/docs/intro/install/
kubectl kustomize / kustomize CLI	Lab 14 (Kustomize overlays)	Built into kubectl 1.14+ (kubectl kustomize)
killer.sh / Killercodea	Mock-exam practice on Day 4	killer.sh (CKAD simulator) · killercodea.com

0.2 Two ways to run every lab

Option A — Killercodea (fastest). Each lab header has a Killercodea link. The environment runs in a browser terminal with a real Kubernetes cluster and Docker pre-installed — nothing to install locally.

Option B — Local cluster. Install Docker Desktop (which includes Kubernetes), or set up minikube or kind. Run `kubectl cluster-info` to verify your cluster is reachable.

0.3 Exam-speed aliases — set these before every session

Set these at the start of each KillerCoda or local session. They save many keystrokes on the real CKAD exam (2 hours, 100% hands-on).

```
alias k=kubectl
export do="--dry-run=client -o yaml"
source <(kubectl completion bash)
complete -o default -F __start_kubectl k
```

Note: On the real CKAD exam these aliases are pre-configured. Practise using them in every lab so the muscle memory is there on exam day.

0.4 Get the lab files

Every lab folder under ``labs/`` is self-contained — it holds the `lab.md` steps plus any YAML manifests, Dockerfiles or Helm charts you need. Work through each folder as you go.

Note: **GitHub repo:** <https://github.com/tertiarycourses/TGS-2025053212-CKAD> · clone it or use **Code** → **Download ZIP**, then `cd` into each lab folder as you go.

DAY 1 — DOMAIN 1: Application Design and Build (Labs 1–8)

Lab 1 — Build a Container Image with Docker

Domain: Application Design and Build · **Topic:** Container images · **Duration:** 45 min
· **KillerCoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

Write a Dockerfile from scratch, build a tagged image, run it locally and inspect its layers. CKAD expects you to read and write Dockerfiles confidently — you may be asked to fix a broken one or produce one under time pressure.

What you'll build

Write a Dockerfile, build and tag an image, run it, then inspect its layers with docker history.

Step 1 — Write the Dockerfile

Each instruction below becomes one cached image layer. Always pin a version tag.

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
```

```
EXPOSE 8080
CMD ["python", "app.py"]
```

Note: `EXPOSE` is documentation only — it does **not** publish the port. `-p host:container` is what actually opens it.

Step 2 — Build and tag the image

```
docker build -t ckad/hello:1.0 .
docker images ckad/hello
```

Step 3 — Run and test the container

```
docker run -d --name hello -p 8080:8080 ckad/hello:1.0
curl http://localhost:8080
```

Step 4 — Inspect the image layers

`docker history` prints one row per Dockerfile instruction — one instruction, one layer.

```
docker history ckad/hello:1.0
docker rm -f hello && docker rmi ckad/hello:1.0
```

✅ **Test it:** `curl http://localhost:8080` returns `hello from CKAD 2026 lab 1`, and `docker history` shows a distinct layer for each Dockerfile instruction.

Lab 2 — Multi-Stage Dockerfile

Domain: Application Design and Build · **Topic:** Container images · **Duration:** 45 min
· **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

Separate the build stage from the runtime stage to produce images that are 10x smaller and contain no compiler toolchain. Multi-stage builds are a CKAD staple — you must be able to write one from scratch and explain why it shrinks the image.

What you'll build

Build a single-stage baseline, then a multi-stage image with `COPY --from` and `distroless`, and compare sizes.

Step 1 — Single-stage baseline (large image)

This image ships the entire Go toolchain (~800 MB) just to run a 6 MB binary.

```
FROM golang:1.22
WORKDIR /src
COPY main.go .
RUN go mod init demo && go build -o app main.go
CMD ["/app"]
```

Step 2 — Multi-stage Dockerfile (small image)

```
# Stage 1: compile
FROM golang:1.22 AS builder
WORKDIR /src
COPY main.go .
RUN go mod init demo && CGO_ENABLED=0 go build -o /out/app main.go

# Stage 2: runtime only
FROM gcr.io/distroless/static-debian12
COPY --from=builder /out/app /app
ENTRYPOINT ["/app"]
```

COPY --from=builder pulls only the compiled binary into the final image; the toolchain stays in the builder stage and is discarded.

Step 3 — Build both and compare sizes

```
docker build -f Dockerfile.single -t demo:single .
docker build -f Dockerfile.multi -t demo:multi .
docker images | grep demo          # ~800MB vs ~8MB
```

Note: Only the **last FROM** stage ends up in the final image. **CGO_ENABLED=0** produces a static binary that runs in distroless/scratch — no shell, smaller attack surface.

✓ **Test it:** `docker images | grep demo` shows `demo:multi` roughly 100x smaller than `demo:single`, and `demo:multi` still serves `hello from multi-stage build`.

Lab 3 — Create and Manage Pods

Domain: Application Design and Build · **Topic:** Pods · **Duration:** 45 min ·

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

The Pod is the smallest schedulable unit in Kubernetes. Create Pods imperatively, generate YAML with `--dry-run`, edit live manifests, and use the exam-critical `$do` alias that saves 30+ seconds per question.

What you'll build

Set exam-speed aliases, run Pods imperatively, scaffold YAML with `$do`, then apply and inspect.

Step 1 — Set exam-speed aliases (run these first on exam day)

```
alias k=kubectl
export do="--dry-run=client -o yaml"
```

Step 2 — Create a Pod imperatively

```
k run web --image=nginx:1.25 --port=80
k get pod web -o wide          # shows node and Pod IP
```

Step 3 — Generate a manifest without creating it

`$do` expands to `--dry-run=client -o yaml` — redirect to a file, edit, then apply. This is the standard CKAD workflow.

```
k run web2 --image=nginx:1.25 --port=80 $do > web2.yaml
k apply -f web2.yaml
k get pod web2 --show-labels
```

Step 4 — Inspect a running Pod

```
k describe pod web
k get pod web -o jsonpath='{.status.podIP}'; echo
```

Step 5 — Override the container command

```
k run sleeper --image=busybox --restart=Never -- sh -c 'sleep 3600'
k delete pod web web2 sleeper --force --grace-period=0
```

Note: `--restart=Never` creates a bare Pod (not a Deployment). Everything after `--` becomes the container command. `--force --grace-period=0` skips the 30s grace period.

✅ **Test it:** `k get pod web2 --show-labels` shows the `tier=frontend` label, and the `--dry-run=client -o yaml` workflow produces a valid manifest you can apply.

Lab 4 — Jobs — Run-to-Completion Workloads

Domain: Application Design and Build · **Topic:** Batch workloads · **Duration:** 45 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

A Job runs Pods until a required number of successful completions is reached, then stops. CKAD tests completions, parallelism, backoffLimit and activeDeadlineSeconds in almost every sitting — you must write a Job manifest from memory.

What you'll build

Create a one-shot Job, a parallel Job with completions, and explore backoffLimit and activeDeadlineSeconds.

Step 1 — One-shot Job (imperative)

```
k create job hello --image=busybox -- echo "hello CKAD 2026"
k logs -l job-name=hello
```


Step 2 — Parallel Job with multiple completions

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pi-parallel
spec:
  completions: 5
  parallelism: 2
  backoffLimit: 4
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: pi
        image: perl:5.34
        command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(50)"]
```

completions: 5 = need five successful Pods · parallelism: 2 = at most two at once ·
backoffLimit: 4 = allow four failures before failing the Job.

Step 3 — Apply and watch completions climb

```
k apply -f parallel.yaml
k get job pi-parallel -w          # COMPLETIONS ticks 0/5 → 5/5
```

Note: `restartPolicy: Never` is **mandatory** in Job Pod templates.
`activeDeadlineSeconds` is a hard wall-clock ceiling — it overrides `backoffLimit`.

✅ **Test it:** `k get job pi-parallel` reaches 5/5 completions with no more than two Pods running simultaneously during the run.

Lab 5 — CronJobs — Scheduled Workloads

Domain: Application Design and Build · **Topic:** Batch workloads · **Duration:** 40 min ·
Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

A CronJob creates Jobs on a time-based schedule using standard cron syntax. CKAD tests `concurrencyPolicy`, `timeZone`, `startingDeadlineSeconds`, history limits, manual triggers and suspend/resume — these fields appear verbatim in exam questions.

What you'll build

Create a CronJob imperatively and declaratively, suspend/resume it, then trigger it manually.

Step 1 — Create a CronJob imperatively

```
k create cronjob date-printer --image=busybox \
  --schedule="*/1 * * * *" -- sh -c "date; echo from cronjob"
k get cronjob date-printer
```

Step 2 — Declarative CronJob with all key fields

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: report
spec:
  schedule: "*/2 * * * *"
  timeZone: "Asia/Singapore"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 2
  startingDeadlineSeconds: 30
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report
              image: busybox
              command: ["sh", "-c", "echo report at $(date)"]
```

Step 3 — Suspend, resume and trigger manually

```
k patch cronjob report -p '{"spec":{"suspend":true}}' # pause
k patch cronjob report -p '{"spec":{"suspend":false}}' # resume
k create job --from=cronjob/report report-manual # run now
```

Note: `concurrencyPolicy`: Allow (default) / Forbid / Replace. `timeZone` is a newer field — always set it. `create job --from=cronjob/<name>` runs the workload immediately.

✓ **Test it:** `k get cronjob report` shows `SUSPEND True` after the patch, and `k create job --from=cronjob/report report-manual` produces an immediate one-off run.

Lab 6 — Multi-Container Pods — Sidecar Pattern

Domain: Application Design and Build · **Topic:** Multi-container Pods · **Duration:** 40 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

Containers in the same Pod share a network namespace and can share volumes. CKAD tests the sidecar pattern (a helper reads/writes a shared volume) and the native sidecar feature (Kubernetes 1.29+). You must exec into and read logs from each container.

What you'll build

Run a sidecar that tails a shared emptyDir log, read each container's logs, then build a native sidecar.

Step 1 — Sidecar sharing a log file via emptyDir

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-sidecar
spec:
  volumes:
    - name: logs
      emptyDir: {}
  containers:
    - name: app
      image: busybox
      command: ["sh", "-c", "while true; do date >> /var/log/app.log; sleep 2; done"]
      volumeMounts:
        - {name: logs, mountPath: /var/log}
    - name: log-shipper
      image: busybox
      command: ["sh", "-c", "tail -F /var/log/app.log"]
      volumeMounts:
        - {name: logs, mountPath: /var/log}
```

Step 2 — Read logs from each container separately

```
k logs app-with-sidecar -c app | head -5
k logs app-with-sidecar -c log-shipper | head -5
k exec -it app-with-sidecar -c app -- sh -c 'wc -l /var/log/app.log'
```

Step 3 — Native sidecar container (Kubernetes 1.29+)

A native sidecar is an **initContainer** with **restartPolicy: Always** — it starts before the main containers and runs for the Pod's lifetime without blocking startup.

```
initContainers:
- name: log-collector
  image: busybox
  restartPolicy: Always
  command: ["sh", "-c", "while true; do echo alive; sleep 5; done"]
```

Note: With multiple containers in a Pod, always specify **-c <name>** for **logs** and **exec**. **emptyDir** is the standard glue volume between sidecar and main containers.

✓ **Test it:** `k logs app-with-sidecar -c log-shipper` shows the same lines `app` is writing, proving both containers share the `emptyDir` volume.

Lab 7 — Init Containers

Domain: Application Design and Build · **Topic:** Multi-container Pods · **Duration:** 40 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

Init containers run in order, to completion, before any main container starts. Use them to seed shared volumes, wait for upstream services, or do one-time setup. CKAD asks you to write init containers and read `Init:N/M` Pod status correctly.

What you'll build

Seed an nginx web root with an init container, then add an init container that waits for a Service via DNS.

Step 1 — Init container that seeds the web root

```
apiVersion: v1
kind: Pod
metadata:
  name: web-init
spec:
  volumes:
  - name: html
    emptyDir: {}
  initContainers:
  - name: seed
    image: busybox
    command: ["sh", "-c", "echo '<h1>Seeded by init</h1>' > /work/index.html"]
    volumeMounts:
    - {name: html, mountPath: /work}
  containers:
  - name: web
    image: nginx:1.25
    volumeMounts:
    - {name: html, mountPath: /usr/share/nginx/html}
```

While the init container runs the Pod shows `Init:0/1`. After it exits 0 the main container starts and status becomes `Running`.

Step 2 — Verify the seeded content

```
k exec web-init -- cat /usr/share/nginx/html/index.html
```

Step 3 — Init container that waits for a Service

```
initContainers:
- name: wait-for-db
```

```
image: busybox
command: ["sh", "-c", "until nslookup db.default.svc.cluster.local; do sleep 2; done"]
```

Note: The Pod stays in **Init:0/1** until DNS resolves. Creating the **db** Service unblocks it. If an init container fails it is restarted per the Pod's restartPolicy.

✓ **Test it:** `k exec web-init -- cat /usr/share/nginx/html/index.html` returns the seeded heading, proving the init container populated the volume before nginx started.

Lab 8 — Volumes — emptyDir and hostPath

Domain: Application Design and Build · **Topic:** Volumes · **Duration:** 40 min ·

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

Containers are ephemeral — data written to the container filesystem is lost on restart. Volumes survive restarts and can be shared between containers. CKAD tests emptyDir, emptyDir.medium: Memory and hostPath, plus mounting strategies.

What you'll build

Share an emptyDir between two containers, mount a RAM-backed tmpfs volume, then mount a node directory with hostPath.

Step 1 — emptyDir shared between two containers

```
volumes:
- name: shared
  emptyDir: {}
containers:
- name: writer
  image: busybox
  command: ["sh", "-c", "echo hello-shared > /data/msg.txt; sleep 3600"]
  volumeMounts: [{name: shared, mountPath: /data}]
- name: reader
  image: busybox
  command: ["sh", "-c", "sleep 5; cat /data/msg.txt; sleep 3600"]
  volumeMounts: [{name: shared, mountPath: /data}]
```

Step 2 — emptyDir backed by RAM (tmpfs)

```
volumes:
- name: fast
  emptyDir:
    medium: Memory
    sizeLimit: 64Mi
```

medium: **Memory** mounts a tmpfs — data lives in RAM, is faster, and disappears on Pod termination. It counts against the container's memory limit.

Step 3 — hostPath: access files on the node

```
volumes:  
- name: etc  
  hostPath:  
    path: /etc  
    type: Directory
```

Note: Volumes are declared under `spec.volumes` and consumed via `spec.containers[].volumeMounts`. Avoid `hostPath` in multi-tenant clusters — it is a security risk; use it only for DaemonSets and node-level tools.

✓ **Test it:** `k logs scratch -c reader` prints `hello-shared`, proving the writer and reader containers share the same `emptyDir` volume.

DAY 2 — DOMAIN 2: Application Deployment (Labs 9–15)

Lab 9 — Deployments and ReplicaSets

Domain: Application Deployment · **Topic:** Deployments & ReplicaSets · **Duration:** 45 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

A Deployment manages a ReplicaSet, which manages Pods. CKAD tests the full ownership chain, scaling, environment-variable injection and reading ReplicaSet history — required to debug failed rollouts.

What you'll build

Create a Deployment imperatively, scale it, generate YAML, inject env vars, then read its ReplicaSet history.

Step 1 — Create a Deployment imperatively

One command creates three resources: Deployment → ReplicaSet (hash-suffixed) → Pods.

```
k create deployment web --image=nginx:1.25 --replicas=3  
k get deploy,rs,pod -l app=web
```

Step 2 — Scale up and down

```
k scale deployment web --replicas=5
k scale deployment web --replicas=2
k get pods -l app=web
```

Step 3 — Generate Deployment YAML and apply

```
k create deployment api --image=httpd:2.4 --replicas=2 $do > api.yaml
k apply -f api.yaml
```

Step 4 — Inject an env var and observe ReplicaSet history

```
k set env deployment/api APP_COLOR=blue
k get rs -l app=api # old RS 0/0/0, new RS 2/2/2
```

Note: `kubectl set env / set image` mutate the Pod template and trigger a new ReplicaSet (a rolling update). Old ReplicaSets are kept for rollback — `revisionHistoryLimit` (default 10) controls how many.

✓ **Test it:** `k get deploy,rs,pod -l app=web` shows one Deployment owning one ReplicaSet owning the Pods, and `k get rs -l app=api` shows two ReplicaSets after the env change.

Lab 10 — Rolling Updates and Rollback

Domain: Application Deployment · **Topic:** Rolling Updates & Rollback · **Duration:** 45 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

A Deployment performs zero-downtime upgrades by gradually replacing Pods one ReplicaSet at a time. CKAD tests maxSurge, maxUnavailable, rollout pause/resume, history inspection and rollback — often as a timed multi-step question.

What you'll build

Tune the rolling-update strategy, trigger an image update, inspect history, pause/resume, then roll back.

Step 1 — Create the initial Deployment

```
k create deployment web --image=nginx:1.24 --replicas=4
k rollout status deployment/web
```

Step 2 — Tune the rolling-update strategy

maxSurge: 1 = at most one extra Pod above desired · maxUnavailable: 1 = at most one Pod down.

```
k patch deployment web -p \
'{"spec":{"strategy":{"rollingUpdate":{"maxSurge":1,"maxUnavailable":1}}}}'
```

Step 3 — Trigger an image update and watch the rollout

```
k set image deployment/web nginx=nginx:1.25
k rollout status deployment/web
k get rs -l app=web          # old RS drains to 0, new RS ramps to 4
```

Step 4 — View history, pause, batch changes, resume

```
k rollout history deployment/web
k rollout pause deployment/web
k set image deployment/web nginx=nginx:1.26
k set env deployment/web APP_ENV=production
k rollout resume deployment/web
```

Step 5 — Roll back

```
k rollout undo deployment/web
k rollout undo deployment/web --to-revision=1
```

Note: Pausing batches multiple mutations into a single new ReplicaSet — one rollout, not two. **rollout undo** reverts to the previous (or a specified) revision instantly.

✓ **Test it:** **k rollout history deployment/web** lists each revision, and **k rollout undo** returns the Deployment to the prior image with zero downtime.

Lab 11 — Blue/Green Deployment

Domain: Application Deployment · **Topic:** Blue/Green Deployment · **Duration:** 40 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

Blue/Green keeps two complete copies of the app alive at once. A Service selector switch flips 100% of traffic from blue to green in one atomic operation — with instant rollback if anything breaks.

What you'll build

Deploy blue (live) and green (idle), smoke-test green, flip the Service selector, then roll back with one patch.

Step 1 — Deploy blue (current production) behind a Service

```
# Deployment web-blue (labels app=web, version=blue) + Service web
# selector: {app: web, version: blue}
k apply -f blue.yaml
k get pods -l app=web --show-labels
```

Step 2 — Deploy green (next version, no live traffic yet)

```
# Deployment web-green (labels app=web, version=green), nginx:1.25
k apply -f green.yaml
k get pods -l app=web --show-labels
```


Step 3 — Smoke-test green directly before cutover

```
GREEN_POD=$(k get pod -l version=green -o jsonpath='{.items[0].metadata.name}')  
k exec $GREEN_POD -- curl -sI localhost:80 | head -2
```

Step 4 — Flip the Service to green (atomic cutover)

```
k patch service web -p '{"spec":{"selector":{"app":"web","version":"green"}}}'  
k describe svc web | grep Selector
```

Step 5 — Roll back instantly if needed

```
k patch service web -p '{"spec":{"selector":{"app":"web","version":"blue"}}}'
```

Note: Cutover and rollback are both a single `kubectl patch service selector` update — atomic, milliseconds, no Pod churn. Blue stays running as an instant fallback.

✓ **Test it:** After the patch, `k describe svc web | grep Selector` shows `version=green`, and patching it back to `version=blue` restores the old version instantly.

Lab 12 — Canary Deployment

Domain: Application Deployment · **Topic:** Canary Deployment · **Duration:** 40 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

A canary release sends a small fraction of live traffic to a new version while the bulk stays on stable. In Kubernetes the split is approximated by replica ratios behind one broad Service selector — no special tooling required.

What you'll build

Run a 9-replica stable and 1-replica canary behind one Service, verify the ~10% split, then promote.

Step 1 — Stable Deployment (90% of traffic)

The Service selector uses only `app: web` — it routes to both stable and canary Pods.

```
# Deployment web-stable: replicas 9, labels app=web,track=stable  
# Service web: selector {app: web}, port 5678  
k apply -f stable.yaml
```

Step 2 — Canary Deployment (10% of traffic)

```
# Deployment web-canary: replicas 1, labels app=web,track=canary  
k apply -f canary.yaml  
k get pods -l app=web --show-labels
```

Step 3 — Verify the traffic split

```
k run probe --image=busybox --restart=Never -it --rm -- sh -c \
  'for i in $(seq 1 30); do wget -qO- web:5678; done | sort | uniq -c'
# ~27 stable, ~3 canary
```

Step 4 — Promote: scale up canary, retire stable

```
k scale deployment web-canary --replicas=9
k scale deployment web-stable --replicas=0
```

Note: Traffic split is approximated by replica ratio (9:1 $\approx$ 90/10). Promotion is just **kubectl scale** on both Deployments — no Service change. For precise percentage splits use a service mesh (Istio/Linkerd) — beyond CKAD scope.

✅ **Test it:** The probe loop returns roughly 90% **stable** and 10% **canary**, and after scaling, all traffic shifts to the canary version with no Service edit.

Lab 13 — Helm — Install, Upgrade, Rollback

Domain: Application Deployment · **Topic:** Helm · **Duration:** 45 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

Helm is the Kubernetes package manager. A chart is a bundle of templated YAML; a release is a deployed instance. CKAD tests install, upgrade, rollback, list and value overrides — all under exam time pressure.

What you'll build

Add a chart repo, install a release with overrides, inspect it, upgrade, roll back, then uninstall.

Step 1 — Add a chart repository

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo bitnami/nginx | head -5
```

Step 2 — Install a release with value overrides

The release name **web** is what you reference for every later command; **--set** overrides chart defaults.

```
helm install web bitnami/nginx \
  --set service.type=ClusterIP --set replicaCount=2
helm list
```

Step 3 — Inspect the generated manifests and values

```
helm get manifest web | head -40
helm get values web
```

Step 4 — Upgrade, then roll back

```
helm upgrade web bitnami/nginx --set replicaCount=4 --reuse-values
helm history web
helm rollback web 1
helm uninstall web
```

Note: `--reuse-values` preserves prior overrides while changing only the new one. `helm history <release>` tracks every revision; rollback creates a new revision rather than deleting history.

✓ **Test it:** `helm history web` lists each revision with its status, and `helm rollback web 1` returns the release to its first revision as a new history entry.

Lab 14 — Kustomize Overlays

Domain: Application Deployment · **Topic:** Kustomize · **Duration:** 45 min ·

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

Kustomize reuses a single base set of manifests and applies environment-specific overlays without templating. It is built into kubectl (-k). CKAD tests namePrefix, commonLabels, images and strategic-merge patches.

What you'll build

Build a base, add dev and prod overlays (prefix, labels, image tag, replica patch), preview, then apply.

Step 1 — Base kustomization (shared by all environments)

```
# base/kustomization.yaml
resources:
- deployment.yaml
- service.yaml
```

Step 2 — Dev overlay (name prefix + common label)

```
# overlays/dev/kustomization.yaml
resources:
- ../../base
namePrefix: dev-
commonLabels:
  env: dev
```

Step 3 — Prod overlay (image bump + replica patch)

```
# overlays/prod/kustomization.yaml
resources:
- ../../base
namePrefix: prod-
commonLabels: {env: prod}
images:
- name: nginx
  newTag: "1.26"
patches:
- path: replica-patch.yaml
```

Step 4 — Preview, then apply

```
kubectl kustomize overlays/prod | grep -E "name:|replicas:|image:"
kubectl apply -k overlays/dev
kubectl apply -k overlays/prod
```

Note: `namePrefix` and `commonLabels` apply to every resource in the overlay. The `images` block overrides tags without editing base files. `apply -k / kubectl kustomize` are built into kubectl — no extra binary.

✓ **Test it:** `kubectl kustomize overlays/prod` shows name `prod-web`, 5 replicas and `nginx:1.26`, while the dev overlay stays at 1 replica and `nginx:1.25`.

Lab 15 — DaemonSets and StatefulSets

Domain: Application Deployment · **Topic:** DaemonSets & StatefulSets · **Duration:** 45 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

A DaemonSet runs exactly one Pod per matching node (log collectors, CNI agents). A StatefulSet gives Pods stable identities and ordered start/stop (databases). CKAD expects you to know when to use each and write the YAML.

What you'll build

Run a DaemonSet on every node, then a StatefulSet with a headless Service, stable DNS and ordered scaling.

Step 1 — DaemonSet on every node

tolerations: operator: Exists schedules onto control-plane nodes too (needed on single-node Killercoda).

```
# DaemonSet node-agent, image busybox
k apply -f ds.yaml
k get ds,pods -l app=node-agent -o wide # DESIRED = node count
```

Step 2 — Headless Service for the StatefulSet

```
# Service db: clusterIP: None, selector app=db, port 5432
k apply -f headless.yaml
```

Step 3 — StatefulSet with stable identities

```
# StatefulSet db: serviceName db, replicas 3
k apply -f sts.yaml
k rollout status sts/db
k get pods -l app=db          # db-0 → db-1 → db-2 in order
```

Step 4 — Verify stable per-Pod DNS and scale

```
nslookup db-0.db.default.svc.cluster.local
k scale sts db --replicas=4    # adds db-3
k scale sts db --replicas=2    # removes db-3 then db-2
```

Note: DaemonSet = one Pod per matching node. StatefulSet = stable names (db-0, db-1) and ordered start/stop; a headless Service (clusterIP: None) gives each Pod its own DNS. Scale-down always removes the highest ordinal first.

✓ **Test it:** `k get pods -l app=db` shows db-0, db-1, db-2 created in order, and `nslookup db-0.db.default.svc.cluster.local` resolves to a single Pod IP.

DAY 3 — DOMAINS 3 & 4: Observability, Maintenance, Configuration & Security (Labs 16–26)

Lab 16 — Liveness, Readiness & Startup Probes

Domain: Observability & Maintenance · **Topic:** Probes · **Duration:** 45 min ·

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

Kubernetes uses three probes to manage health: livenessProbe restarts a failed container, readinessProbe removes it from Service endpoints, startupProbe gives slow apps time to boot. CKAD tests all three handlers (httpGet, tcpSocket, exec) and timing.

What you'll build

Add HTTP readiness+liveness probes, trigger a liveness restart, add a TCP probe, then a startupProbe with exec.

Step 1 — HTTP liveness + readiness probe

```
readinessProbe:
  httpGet: {path: /, port: 80}
```

```
  initialDelaySeconds: 2
  periodSeconds: 5
livenessProbe:
  httpGet: {path: /, port: 80}
  initialDelaySeconds: 10
  periodSeconds: 10
  failureThreshold: 3
```

Step 2 — Trigger a liveness failure (watch RESTARTS climb)

```
k exec web-probes -- rm /usr/share/nginx/html/index.html
sleep 35
k get pod web-probes
```

Step 3 — TCP socket probe

```
readinessProbe:
  tcpSocket: {port: 6379}
  periodSeconds: 5
```

Step 4 — startupProbe for slow-starting apps

startupProbe runs exclusively until it succeeds: $\text{failureThreshold } 30 \times \text{periodSeconds } 5 = 150\text{s}$ budget.

```
startupProbe:
  exec: {command: ["cat", "/tmp/ready"]}
  failureThreshold: 30
  periodSeconds: 5
```

Note: Three handlers — httpGet, tcpSocket, exec. readinessProbe failure removes the Pod from Service endpoints without restarting it; livenessProbe failure restarts the container; startupProbe blocks the other two until the app is initialised.

✅ **Test it:** Deleting the index page makes the liveness probe fail three times and the container restarts (RESTARTS increments), while the TCP and startup Pods reach READY 1/1.

Lab 17 — Container Logging

Domain: Observability & Maintenance · **Topic:** Logging · **Duration:** 40 min ·

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

kubectl logs is the first debugging tool on the exam. Read logs from single and multi-container Pods, follow a live stream, retrieve logs from a crashed container, and aggregate logs across a Deployment.

What you'll build

Read and tail logs, stream with `-f`, recover crash logs with `--previous`, then aggregate by container and label.

Step 1 — Read recent lines and follow a live stream

```
k logs noisy --tail=10
k logs -f noisy &          # -f streams like tail -f
kill %1
```

Step 2 — Retrieve logs from a crashed container

`--previous` reads the prior terminated container — the key tool for CrashLoopBackOff.

```
k logs crasher --previous | head -5
```

Step 3 — Multi-container Pod logs

```
k logs multi -c app | head -3
k logs multi -c sidecar | head -3
k logs multi --all-containers=true --prefix=true | head -8
```

Step 4 — Aggregate logs across a Deployment

```
k logs deployment/fleet --tail=3
k logs -l app=fleet --prefix=true --tail=2
```

Note: `--tail=N` for recent lines, `-f` to stream, `--previous` for crash loops, `-c <name>` for a specific container, `-l <selector>` to aggregate matching Pods. `--prefix=true` labels each line with `[pod/container]`.

✓ **Test it:** `k logs crasher --previous` shows the message printed before the crash, and `k logs -l app=fleet --prefix=true` interleaves lines from all three Pods.

Lab 18 — kubectl top and Metrics Server

Domain: Observability & Maintenance · **Topic:** Metrics · **Duration:** 35 min ·

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

`kubectl top` shows real-time CPU and memory for nodes and Pods. It requires the metrics-server add-on. CKAD tests installing metrics-server, reading node/Pod metrics, and sorting by resource usage.

What you'll build

Install metrics-server, read node metrics, generate CPU load, then sort Pod metrics by CPU and memory.

Step 1 — Install metrics-server (and trust kubelet certs on KillerCoda)

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
components.yaml
kubectl patch -n kube-system deployment metrics-server --type=json -p='
[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--
kubelet-insecure-tls"}]'
until kubectl top node 2>/dev/null; do sleep 5; done
```

Step 2 — Top nodes

```
k top node          # CPU cores/% and memory bytes/% per node
```

Step 3 — Generate load, then top Pods sorted

```
k run cpu-burner --image=busybox -- sh -c 'while true; do ;; done'
sleep 30
k top pod --sort-by=cpu
k top pod --sort-by=memory
k top pod -A --sort-by=cpu | head -10
```

Note: `kubectl top` needs metrics-server running. `--kubelet-insecure-tls` is required on KillerCoda's self-signed certs. `cpu-burner` should top the CPU sort; `-A` spans all namespaces.

✓ **Test it:** After ~30s, `k top node` reports CPU/memory and `k top pod --sort-by=cpu` lists `cpu-burner` at the top of the usage ranking.

Lab 19 — Debugging Pods and Events

Domain: Observability & Maintenance · **Topic:** Debugging · **Duration:** 45 min ·

KillerCoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

CKAD regularly includes broken workloads you must diagnose and fix under time pressure. Identify and resolve ImagePullBackOff, CrashLoopBackOff and OOMKilled, and use `kubectl debug` for live triage.

What you'll build

Diagnose three classic failures with `describe/logs/events`, then inject an ephemeral debug container.

Step 1 — ImagePullBackOff (wrong image name)

```
k describe pod typo | tail -15          # 'Failed to pull image'
k delete pod typo --force --grace-period=0
k run typo --image=nginx:1.25           # corrected
```


Step 2 — CrashLoopBackOff (container exits non-zero)

```
k describe pod crash | grep -A3 "Last State"
k logs crash --previous
```

Step 3 — OOMKilled (exceeds memory limit)

```
k describe pod hungry | grep -A4 "Last State"    # Reason: OOMKilled
# fix: raise resources.limits.memory
```

Step 4 — Cluster-wide events + ephemeral debug container

```
k get events -A --sort-by=.lastTimestamp | tail -20
k get events --field-selector type=Warning
k debug crash -it --image=busybox --target=crash -- sh
```

Note: ImagePullBackOff → wrong image/tag. CrashLoopBackOff → use `logs --previous`. OOMKilled → raise the memory limit. `kubectl debug` injects an ephemeral container sharing the target's namespaces — vital for distroless images with no shell.

✓ **Test it:** `k describe` reveals the correct failure reason for each Pod (ImagePullBackOff, CrashLoopBackOff, OOMKilled), and `k debug` opens a shell inside the running Pod.

Lab 20 — API Deprecations & kubectl explain

Domain: Observability & Maintenance · **Topic:** API Deprecations · **Duration:** 35 min
· **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

The Kubernetes API evolves — old versions are deprecated and removed. CKAD tests `kubectl explain`, `api-resources` and `api-versions`. You must find the correct `apiVersion` for any resource on exam day using only `kubectl`.

What you'll build

List resources and versions, explore schemas with `explain`, detect a removed API, and look up the current version.

Step 1 — List resources, groups and served versions

```
k api-resources --api-group=apps
k api-resources --api-group=batch
k api-versions | sort
```

Step 2 — Explore a schema with explain

`--recursive` prints the full field tree — use it when you forget a nested field path.

```
k explain deployment.spec.strategy.rollingUpdate
k explain pod.spec.containers.resources --recursive | head -30
```

Step 3 — Detect a removed API and find the replacement

```
# extensions/v1beta1 Ingress was removed in 1.22
k explain ingress --api-version=networking.k8s.io/v1 | head -10
k api-resources | grep -i ingress
```

Step 4 — Stable apiVersions to memorise (v1.35)

```
Pod/Service/ConfigMap/Secret/SA/Quota/LimitRange -> v1
Deployment/StatefulSet/DaemonSet/ReplicaSet      -> apps/v1
Job/CronJob                                       -> batch/v1
Ingress/NetworkPolicy                           -> networking.k8s.io/v1
Role/RoleBinding/ClusterRole(+Binding)           ->
rbac.authorization.k8s.io/v1
HorizontalPodAutoscaler                          -> autoscaling/v2
```

Note: `kubectl api-resources` shows the correct apiVersion in its APIVERSION column — the exam-legal way to look it up. `kubectl explain <res>.<field>` documents fields without leaving the terminal.

✓ **Test it:** `k api-resources | grep -i ingress` shows `networking.k8s.io/v1`, and `k explain deployment.spec.strategy.rollingUpdate` prints the `maxSurge/maxUnavailable` fields.

Lab 21 — ConfigMaps — Env & Volume Injection

Domain: Configuration & Security · **Topic:** ConfigMaps · **Duration:** 45 min ·

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

ConfigMaps inject non-secret configuration into Pods. CKAD tests all three injection styles: a single env var (`configMapKeyRef`), bulk env vars (`envFrom`), and file-based volume mounts — and that only file mounts update live.

What you'll build

Create ConfigMaps three ways, inject one key, inject all keys with `envFrom`, mount as files, then update live.

Step 1 — Create ConfigMaps three ways

```
k create configmap app-cfg --from-literal=COLOR=blue --from-literal=GREETING=hello
k create configmap app-conf --from-file=app.conf
k create configmap app-env --from-env-file=env.list
```

Step 2 — Inject a single key as an env var

```
env:
- name: COLOR
  valueFrom:
    configMapKeyRef: {name: app-cfg, key: COLOR}
```

Step 3 — Inject all keys with envFrom

```
envFrom:
- configMapRef: {name: app-cfg}
- configMapRef: {name: app-env}
```

Step 4 — Mount a ConfigMap as a file volume

```
volumeMounts:
- {name: cfg, mountPath: /etc/app}
volumes:
- name: cfg
  configMap: {name: app-conf}
```

Note: Creation: `--from-literal`, `--from-file`, `--from-env-file`.

Consumption: `configMapKeyRef` (one key), `envFrom` (all keys), volume mount (file per key). Volume-mounted ConfigMaps update live (~60s); env-var injections are fixed at Pod startup and need a restart.

✅ **Test it:** The single-key Pod logs `COLOR=blue`, the envFrom Pod shows all four variables, and the mounted ConfigMap appears as files under `/etc/app`.

Lab 22 — Secrets

Domain: Configuration & Security · **Topic:** Secrets · **Duration:** 45 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

Secrets are like ConfigMaps but base64-encoded with stricter access. CKAD tests generic, tls and docker-registry types, env-var injection, file mounts with defaultMode, and decoding values. Secrets are not encrypted — protect them with RBAC.

What you'll build

Create a generic Secret, decode it, inject as env vars, mount with 0400 mode, then create tls and docker-registry Secrets.

Step 1 — Create a generic Secret and decode a value

```
k create secret generic db-cred \
  --from-literal=DB_USER=admin --from-literal=DB_PASS=s3cr3t
k get secret db-cred -o jsonpath='{.data.DB_PASS}' | base64 -d; echo
```

Step 2 — Inject Secret keys as env vars

```
envFrom:  
- secretRef: {name: db-cred}
```

Step 3 — Mount a Secret as files with restrictive mode

defaultMode: 0400 = owner read-only. CKAD frequently asks you to set this.

```
volumes:  
- name: s  
  secret:  
    secretName: db-cred  
    defaultMode: 0400
```

Step 4 — TLS and docker-registry Secrets

```
k create secret tls demo-tls --cert=tls.crt --key=tls.key  
k create secret docker-registry myreg \  
  --docker-server=registry.example.com --docker-username=u \  
  --docker-password=p --docker-email=u@example.com
```

Note: Types: `generic`, `tls` (kubernetes.io/tls), `docker-registry`. `envFrom: secretRef` injects all keys; `secretKeyRef` injects one. Reference a registry Secret via `spec.imagePullSecrets`. Base64 ≠ encryption — restrict with RBAC.

✓ **Test it:** `base64 -d` on `.data.DB_PASS` returns `s3cr3t`, the mounted Secret files are mode `0400`, and the `tls` Secret reports type `kubernetes.io/tls`.

Lab 23 — SecurityContext

Domain: Configuration & Security · **Topic:** SecurityContext · **Duration:** 45 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

`securityContext` controls the identity and privileges of containers. CKAD asks you to enforce non-root execution, a read-only root filesystem and dropped Linux capabilities — at Pod level (all containers) or container level (one override).

What you'll build

Run as a set user/group, enforce `runAsNonRoot`, lock the root filesystem, then drop all capabilities.

Step 1 — Run as a specific user and group (Pod level)

```
securityContext:  
  runAsUser: 1000  
  runAsGroup: 3000  
  fsGroup: 2000      # applies to volume mounts
```

Step 2 — Enforce runAsNonRoot (blocks root images)

```
securityContext:
  runAsNonRoot: true
# nginx runs as root -> Pod refuses to start
```

Step 3 — Read-only root filesystem (container level)

```
securityContext:
  readOnlyRootFilesystem: true
# mount an emptyDir for any writable path (e.g. /tmp)
```

Step 4 — Drop Linux capabilities (hardened container)

```
securityContext:
  capabilities:
    drop: ["ALL"]
    add: ["NET_BIND_SERVICE"]
  allowPrivilegeEscalation: false
```

Note: Pod-level securityContext applies to all containers; container-level overrides one. The CKAD hardened pattern = `runAsNonRoot: true` + `readOnlyRootFilesystem: true` + `capabilities.drop: [ALL]` + `allowPrivilegeEscalation: false`.

✅ **Test it:** The non-root Pod logs `uid=1000 gid=3000`, the `runAsNonRoot` nginx Pod refuses to start, and writes to the read-only root filesystem are rejected.

Lab 24 — ServiceAccounts

Domain: Configuration & Security · **Topic:** ServiceAccounts · **Duration:** 40 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

Every Pod runs as a ServiceAccount; the default one has minimal RBAC. CKAD tests creating dedicated ServiceAccounts, attaching them to Pods, disabling the auto-mounted token, and requesting short-lived tokens with `kubectl create token`.

What you'll build

Create a ServiceAccount, attach it to a Pod, inspect the projected token, disable auto-mount, then mint a token.

Step 1 — Create a ServiceAccount and attach it to a Pod

```
k create serviceaccount app-sa
# Pod spec:
spec:
  serviceAccountName: app-sa
k get pod app -o jsonpath='{.spec.serviceAccountName}'; echo
```

Step 2 — Inspect the auto-mounted token inside the Pod

Three files are projected: token (short-lived JWT), ca.crt, namespace.

```
k exec app -- ls /var/run/secrets/kubernetes.io/serviceaccount/
```

Step 3 — Disable token auto-mount (least privilege)

```
k patch sa app-sa -p '{"automountServiceAccountToken": false}'
```

Step 4 — Request a short-lived ad-hoc token

```
TOKEN=$(k create token app-sa --duration=1h)
echo $TOKEN | cut -c1-60
```

Note: `spec.serviceAccountName` attaches a custom SA. Token, CA and namespace are projected under `/var/run/secrets/kubernetes.io/serviceaccount/`. `automountServiceAccountToken: false` enforces least privilege; `kubectl create token` replaces the old Secret-backed tokens (removed in 1.24+).

✓ **Test it:** `k get pod app -o jsonpath='{.spec.serviceAccountName}'` returns `app-sa`, and `kubectl create token app-sa` prints a short-lived JWT.

Lab 25 — RBAC — Roles & RoleBindings

Domain: Configuration & Security · **Topic:** RBAC · **Duration:** 45 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

RBAC controls who can do what on which resources. CKAD tests creating Roles, ClusterRoles, RoleBindings and ClusterRoleBindings imperatively, and validating with `kubectl auth can-i --as`. Know namespace- vs cluster-scope.

What you'll build

Create a Role + RoleBinding for a ServiceAccount, validate with `can-i`, then a ClusterRole bound two ways.

Step 1 — Role: namespace-scoped permissions

```
k create role pod-reader \
  --verb=get,list,watch --resource=pods -n dev
```

Step 2 — RoleBinding: link Role to a ServiceAccount

```
k create rolebinding viewer-binding \
  --role=pod-reader --serviceaccount=dev:viewer -n dev
```

Step 3 — Validate with `kubectl auth can-i`

```
k auth can-i list pods -n dev --as=system:serviceaccount:dev:viewer #
yes
```

```
k auth can-i delete pods -n dev --as=system:serviceaccount:dev:viewer # no
k auth can-i list pods -n default --as=system:serviceaccount:dev:viewer # no
```

Step 4 — ClusterRole bound cluster-wide vs to one namespace

```
k create clusterrole node-reader --verb=get,list --resource=nodes
k create clusterrolebinding nodes-binding \
  --clusterrole=node-reader --serviceaccount=dev:nodes-sa # all namespaces
k create rolebinding dev-node-reader \
  --clusterrole=node-reader --serviceaccount=dev:viewer -n dev # only dev
```

Note: Role + RoleBinding = namespace-scoped. ClusterRole + ClusterRoleBinding = cluster-wide. ClusterRole + RoleBinding = the cluster role's verbs limited to one namespace. `kubectl auth can-i <verb> <res> --as=<identity>` validates without logging in.

✓ **Test it:** `k auth can-i list pods -n dev --as=system:serviceaccount:dev:viewer` returns **yes** while `delete pods` and cross-namespace `list` return **no**.

Lab 26 — ResourceQuota & LimitRange

Domain: Configuration & Security · **Topic:** Quotas & Limits · **Duration:** 40 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

ResourceQuota caps the total resources used across a namespace; LimitRange enforces per-container defaults and maximums. CKAD tests both — write the YAML, apply them, and understand the rejection error messages.

What you'll build

Apply a ResourceQuota and a LimitRange, watch defaults get injected, then violate the max and the quota.

Step 1 — ResourceQuota: namespace-wide ceilings

```
spec:
  hard:
    pods: "5"
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
```

Step 2 — LimitRange: per-container defaults and maximums

```
spec:
  limits:
    - type: Container
```

```
default:      {cpu: 200m, memory: 256Mi}
defaultRequest: {cpu: 100m, memory: 128Mi}
max:          {cpu: 500m, memory: 512Mi}
```

Step 3 — Defaults injected; max enforced

```
k run a --image=nginx:1.25 -n team-a
k get pod a -n team-a -o jsonpath='{.spec.containers[0].resources}'; echo
# a container requesting cpu:1 is rejected: 'maximum cpu per Container is 500m'
```

Step 4 — Exceed the quota

```
for i in 1 2 3 4; do k run quota-$i --image=nginx:1.25 -n team-a; done
k run quota-5 --image=nginx:1.25 -n team-a    # 'exceeded quota: team-a-quota'
k describe quota team-a-quota -n team-a      # Used vs Hard
```

Note: ResourceQuota limits aggregate usage; exceeding it rejects new Pods. LimitRange enforces per-container min/max/defaults. Without a LimitRange, Pods with no resource block cannot be created in a quota-enforced namespace.

✅ **Test it:** A Pod with no resource block gets requests/limits injected by the LimitRange, the 6th Pod is rejected with **exceeded quota**, and **describe quota** shows Used vs Hard.

DAY 4 (½ day) — DOMAIN 5: Services & Networking (Labs 27–30)

Lab 27 — Services — ClusterIP, NodePort, LoadBalancer

Domain: Services & Networking · **Topic:** Services · **Duration:** 45 min · **Killercoda:** <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

A Service gives a stable virtual IP and DNS name to a set of Pods. CKAD requires fluency with ClusterIP, NodePort and LoadBalancer types, kubectl expose, endpoint debugging and the selector-mismatch pattern.

What you'll build

Expose a Deployment as ClusterIP, NodePort and LoadBalancer, inspect EndpointSlices, then debug a selector mismatch.

Step 1 — Backend Deployment + ClusterIP Service

A ClusterIP is reachable only by other Pods in the cluster, not from outside.


```
k create deployment web --image=nginx:1.25 --replicas=3
k expose deployment web --port=80 --target-port=80 --name=web-cip
k describe svc web-cip | grep -E "IP:|Endpoints:"
```

Step 2 — NodePort (a port on every node)

```
k expose deployment web --port=80 --target-port=80 --type=NodePort --name=web-np
PORT=$(k get svc web-np -o jsonpath='{.spec.ports[0].nodePort}')
curl -s http://localhost:$PORT | head -3 # 30000-32767
```

Step 3 — LoadBalancer + EndpointSlices

```
k expose deployment web --port=80 --type=LoadBalancer --name=web-lb
k get endpointslices -l kubernetes.io/service-name=web-cip
```

Step 4 — Debug a selector mismatch (the #1 Service bug)

```
k patch svc web-cip -p '{"spec":{"selector":{"app":"does-not-exist"}}}'
k get endpoints web-cip # empty = no Pod matches
k patch svc web-cip -p '{"spec":{"selector":{"app":"web"}}}'
```

Note: ClusterIP = in-cluster only; NodePort = a port on every node; LoadBalancer = a cloud LB. Empty **Endpoints** almost always means the selector does not match Pod labels. EndpointSlices (v1.21+) succeed Endpoints.

✅ **Test it:** `k describe svc web-cip` lists three endpoint IPs, the NodePort serves the page on `localhost:$PORT`, and a bad selector empties the Endpoints until it is fixed.

Lab 28 — Service DNS

Domain: Services & Networking · **Topic:** Service DNS · **Duration:** 40 min ·

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

CoreDNS gives every Service a stable DNS name:

`<service>.<namespace>.svc.cluster.local`. CKAD tests cross-namespace resolution, Pod DNS records, headless Services, and reading `/etc/resolv.conf` to understand ndots:5.

What you'll build

Resolve a Service by short name and FQDN, cross namespaces, read a Pod DNS record and `/etc/resolv.conf`, then a headless Service.

Step 1 — Resolve from the same namespace (short name)

```
k -n app run client --image=busybox --restart=Never -it --rm -- \
sh -c 'nslookup web; wget -q0- web | head -3'
```

Step 2 — Resolve from a different namespace

Cross-namespace lookups need at least <service>.<namespace>; the FQDN always works.

```
nslookup web                # fails cross-namespace
nslookup web.app            # works
nslookup web.app.svc.cluster.local
```

Step 3 — Pod DNS record and resolv.conf

```
# Pod A-record: <dashed-ip>.<namespace>.pod.cluster.local
cat /etc/resolv.conf
# search app.svc.cluster.local svc.cluster.local cluster.local
# ndots:5
```

Step 4 — Headless Service returns all Pod IPs

```
k -n app create service clusterip headless --clusterip="None" --tcp=80:80
nslookup headless.app      # returns every Pod IP, not one VIP
```

Note: FQDN = <service>.<namespace>.svc.cluster.local. Short names resolve via the search list within the namespace; cross-namespace needs <service>.<namespace>. **ndots:5** is why short names check the search list before an absolute lookup. Headless (clusterIP: None) returns per-Pod IPs.

✓ **Test it:** `nslookup web` works in-namespace but needs `web.app` cross-namespace, and the headless Service resolves to all backing Pod IPs.

Lab 29 — Ingress with TLS

Domain: Services & Networking · **Topic:** Ingress & TLS · **Duration:** 45 min ·

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

An Ingress provides Layer-7 HTTP/HTTPS routing: hostname and path rules forwarding to backend Services. CKAD tests installing a controller, TLS Secrets, host routing and path-based routing with pathType.

What you'll build

Install ingress-nginx, create a TLS Secret, write a host+TLS Ingress, test HTTPS, then add a path-based rule.

Step 1 — Backend Service + TLS Secret

```
k create deployment web --image=hashicorp/http-echo --replicas=2 --
-text=hello-ingress
k expose deployment web --port=5678 --target-port=5678
k create secret tls demo-tls --cert=tls.crt --key=tls.key
```

Step 2 — Ingress with TLS and host routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: {name: demo}
spec:
  ingressClassName: nginx
  tls:
  - hosts: [demo.local]
    secretName: demo-tls
  rules:
  - host: demo.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: {service: {name: web, port: {number: 5678}}}
```

Step 3 — Test HTTPS routing

```
curl -k --resolve demo.local:$HTTPS_PORT:127.0.0.1 \
  https://demo.local:$HTTPS_PORT/ # hello-ingress
```

Step 4 — Add path-based routing

```
# add a second rule: path /v2 -> service v2:5678 (pathType: Prefix)
curl -k --resolve demo.local:$HTTPS_PORT:127.0.0.1 \
  https://demo.local:$HTTPS_PORT/v2 # hello-v2
```

Note: Ingress `apiVersion` is `networking.k8s.io/v1`. `ingressClassName` selects the controller; `tls.secretName` must point at a `kubernetes.io/tls` Secret; `pathType: Prefix` matches a path and everything below it. `--resolve + -k` let curl test without real DNS or a valid cert.

✅ **Test it:** `curl https://demo.local:$HTTPS_PORT/` returns `hello-ingress` over TLS, and the `/v2` path returns `hello-v2` after the second rule is added.

Lab 30 — NetworkPolicy

Domain: Services & Networking · **Topic:** NetworkPolicy · **Duration:** 45 min ·
Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Goal

By default every Pod can talk to every other Pod. NetworkPolicies allow-list specific sources and destinations. CKAD regularly includes a NetworkPolicy question — you must write a default-deny and a selective-allow policy from scratch.

What you'll build

Apply default-deny ingress, a selective podSelector allow, an egress DNS-only lockdown, then a cross-namespace allow.

Step 1 — Default deny: block all ingress

podSelector: {} matches all Pods; an empty ingress list means no traffic is allowed in.

```
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

Step 2 — Selective allow: only role=allowed reaches the backend

```
spec:
  podSelector:
    matchLabels: {app: backend}
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector:
        matchLabels: {role: allowed}
    ports:
    - {protocol: TCP, port: 5678}
```

Step 3 — Egress lockdown: allow DNS only

```
spec:
  podSelector: {matchLabels: {role: blocked}}
  policyTypes: [Egress]
  egress:
  - to:
    - namespaceSelector: {}
      podSelector: {matchLabels: {k8s-app: kube-dns}}
    ports: [{protocol: UDP, port: 53}, {protocol: TCP, port: 53}]
```

Step 4 — Cross-namespace allow (namespaceSelector)

```
ingress:
- from:
  - namespaceSelector:
      matchLabels: {purpose: trusted}
```

Note: Default-deny = `podSelector: {}` + `policyTypes: [Ingress]` with no ingress list. `podSelector` in `from` matches Pods by label in the same namespace; `namespaceSelector` matches whole namespaces. Policies are additive — multiple policies combine with logical OR.

✅ **Test it:** After the allow policy, `client-ok` (role=allowed) reaches the backend while `client-bad` stays blocked, proving the NetworkPolicy is enforced.

Troubleshooting Cheat-Sheet

Symptom	Likely cause & fix
ImagePullBackOff	Wrong image name or tag. Check <code>kubectl describe pod <name></code> Events, fix the image field and recreate.
CrashLoopBackOff	Container exits non-zero. Read <code>kubectl logs <pod> --previous</code> to see what failed.
OOMKilled	Container exceeded its memory limit. Raise <code>resources.limits.memory</code> or reduce the workload.
Pending Pod	<code>kubectl describe pod <name></code> — usually no schedulable node, a missing taint toleration, or an unbound PVC.
Init:0/1 never progresses	Init container is failing or waiting. <code>kubectl logs <pod> -c <init-container-name></code> to diagnose.
Service returns nothing / times out	The Service <code>selector</code> must exactly match the Pod <code>labels</code> . Check <code>kubectl get endpoints <svc></code> .
PVC stuck Pending	No PV matches the claim — check <code>storageClassName</code> , <code>accessModes</code> and requested size.
<code>kubectl top</code> — metrics not available	metrics-server is not running. On KillerCoda patch with <code>--kubelet-insecure-tls</code> .
Error from server (Forbidden)	RBAC is denying access. Check the ServiceAccount, Role and RoleBinding with <code>kubectl auth can-i</code> .
<code>connection refused</code> on NodePort	NodePort range 30000–32767. Confirm the port with <code>kubectl get svc</code> and access via the node IP.
Ingress returns 404 / 503	No matching rule (host/path mismatch) or the backend Service is down. <code>kubectl describe ingress <name></code> .
NetworkPolicy blocks expected traffic	Policies are additive deny-then-allow. Check <code>podSelector</code> and namespace labels with <code>kubectl get ns --show-labels</code> .

Glossary

Term	Meaning
Pod	The smallest Kubernetes unit — one or more containers sharing the same network namespace and volumes.
Deployment	Manages a desired number of identical Pod replicas; self-heals, scales and rolls out updates.
ReplicaSet	The low-level object a Deployment creates to maintain the correct Pod count.
Job	Runs Pods until a set number of successful completions, then stops — used for batch work.
CronJob	Creates Jobs on a cron schedule (<code>*/1 * * *</code> * = every minute).
Service (ClusterIP)	A stable virtual IP + DNS name in front of matching Pods; in-cluster only.
Service (NodePort)	Opens a port (30000–32767) on every node so

	Pods are reachable from outside the cluster.
Service (LoadBalancer)	Asks the cloud provider for an external IP; builds on NodePort.
Ingress	Layer-7 HTTP/HTTPS routing; host- and path-based rules forwarding to backend Services.
NetworkPolicy	Allow-list for Pod-to-Pod traffic (ingress and/or egress); default is open.
ConfigMap	Non-secret key/value data injected into Pods via env vars or volume mounts.
Secret	Like ConfigMap but base64-encoded; access controlled by RBAC. Not encrypted by default.
securityContext	Sets the user, group and Linux capabilities a container runs with.
ServiceAccount	The API identity a Pod presents to the Kubernetes API server.
Role / ClusterRole	List of allowed verbs (get, list, create...) on resources; scoped to namespace or cluster-wide.
RoleBinding / ClusterRoleBinding	Attaches a Role to a subject (ServiceAccount, user, group).
ResourceQuota	Caps the aggregate resources (CPU, memory, pods...) a namespace can consume.
LimitRange	Enforces per-container resource defaults, minimums and maximums within a namespace.
PersistentVolume (PV)	A piece of storage provisioned in the cluster.
PersistentVolumeClaim (PVC)	A request for storage; binds to a matching PV.
Helm	The Kubernetes package manager — installs and upgrades applications from charts.
Kustomize	Layer YAML overlays on a base without templating; built into kubectl apply -k .
DaemonSet	Ensures one Pod runs on every (matching) node — used for log agents, monitoring.
StatefulSet	Like Deployment but with stable Pod names, ordered startup and sticky storage.
emptyDir	A Pod-scoped temporary volume; lives and dies with the Pod; shared by all containers.
hostPath	Mounts a directory from the host node — security risk in multi-tenant clusters.
kubectl --dry-run=client -o yaml	Generates a manifest without creating the resource (\$do alias in labs).
killer.sh	The official CKAD exam simulator — 25 realistic scenario tasks in 2 hours.

Congratulations — you've worked through all five CKAD exam domains across 30 hands-on labs. Keep practising on killer.sh and Killercoda before your exam day.